

---

# In-Context Learning Performance of Hierarchical Networks (H-Nets)

---

Atharv Sampath\*   Tvisha Londhe\*   Tiffany Dang\*   Michael Ofengenden\*

UC Berkeley

{asampath, tvisha.londhe, tiffanyd, mofengenden}@berkeley.edu

## Abstract

We study the in-context learning (ICL) behaviour of Hierarchical Networks (H-Nets), tokenizer-free sequence models that replace fixed subword vocabularies with a learned dynamic chunking mechanism over bytes. An H-Net consists of Mamba-2 encoder-decoder “chunkers” that segment and compress the byte stream into variable-length latent chunks, coupled to an attention-based main network that performs sequence modeling over these learned units. Stacking multiple chunking stages yields deeper hierarchies, where lower stages act as adaptive compressors and higher stages operate on increasingly abstract representations. Using the Mechanistic Architecture Design (MAD) benchmark, we compare flattened, 1-stage, and 2-stage H-Nets under an iso-parameter setting on synthetic ICL tasks including compression, (fuzzy/noisy) in-context recall, memorization, and selective copy. All three variants solve the compression task perfectly (100% accuracy). The flattened H-Net achieves the best overall MAD score (84.9 vs. 81.4 and 82.8 for 1- and 2-stage H-Nets), while the 2-stage H-Net attains the strongest noisy-recall performance (99.7% vs. 96.6% for the flat baseline), suggesting that deeper hierarchies better denoise and stabilize retrieved content. Our results highlight a trade-off: dynamic chunking and multi-stage compression can improve robustness to noise and distributional shift, but may slightly degrade exact token-level recall compared to a flat architecture. Code is available at: <https://github.com/tvisha78/182H-Nets>

## 1 Introduction

Large language models (LLMs) have transformed modern AI in large part due to their ability to perform *in-context learning* (ICL)—that is, to adapt to new tasks given only a few examples in the prompt, without any parameter updates [1, 22]. This capability, first highlighted by GPT-3 [1], tends to improve dramatically with scale, with larger models exhibiting sharp emergent gains in few-shot performance [1, 22]. *In-context* adaptation has become a defining feature of foundation models. However, the mechanisms behind ICL remain an active research question: some studies suggest that models may only appear to learn new tasks by exploiting spurious patterns or existing knowledge [24], while others find that sufficiently large models can flexibly implement novel input-output mappings outside their pretraining distribution [23]. These debates notwithstanding, it is widely recognized that improving a model’s ability to interpret and utilize prompts is crucial for broad generalization.

At the same time, virtually all contemporary LLMs rely on a fixed, subword tokenization preprocessing step (e.g. byte-pair encoding) to convert raw text into model inputs [2]. While convenient, fixed tokenization introduces a **representational bottleneck**: rigid boundaries often misalign with the true semantic or syntactic units of language, and small input perturbations can unpredictably alter the token sequence [4, 3]. These limitations are especially severe in domains such as source

code, genomics, or morphologically rich languages, where natural information units vary widely in length and form [11].

**Hierarchical Networks (H-Nets)** address these challenges with a fully end-to-end architecture that replaces external tokenizers with a learned, dynamic segmentation mechanism [10]. An H-Net processes raw byte sequences through a hierarchy of layers: lower levels handle fine-grained local patterns, while higher layers operate over adaptive chunks trained jointly with the language modeling objective. While H-Nets have demonstrated strong performance in general language modeling [10], their efficacy in *in-context learning*—where the model must rapidly adapt to new patterns in the prompt—remains unexplored. Despite growing progress in tokenizer-free modeling, little is known about how such **hierarchical architectures** perform specifically on *in-context learning* tasks. This question is particularly relevant given the rapid development of efficient architectures—such as State Space Models (SSMs) and linear attention variants—that aim to handle long contexts at lower complexity [13, 12]. While these inductive biases offer efficiency, they sometimes struggle with complex recall behaviors compared to standard Transformers [19]. In this work, we examine where H-Nets fall on this spectrum. We use the Mechanistic Architecture Design (MAD) framework [20] to evaluate H-Nets across a diverse suite of synthetic ICL tasks, comparing them against strong baselines including Mamba, DeltaNet, Hyena, and Transformers. Through these experiments, we aim to determine whether *learned dynamic chunking* provides a beneficial inductive bias for few-shot generalization.

## 2 Related Work

**Tokenization, Tokenizer-Free Modeling, and Dynamic Segmentation.** Most large language models still depend on fixed subword tokenization schemes such as BPE [2], which offer compact vocabularies and efficient training pipelines but impose rigid boundaries that may not match the semantic or syntactic structure of natural language [4]. This misalignment can fragment coherent units across tokens and create spurious sensitivity to shallow surface changes such as spelling variations, Unicode artifacts, or alternative whitespace patterns [3]. The resulting “tokenization bottleneck” is especially severe for domains like source code, DNA, and morphologically rich languages where meaningful units span a wide range of lengths and internal patterns [4, 11]. Tokenizer-free models such as ByT5 [5] avoid these issues by operating directly on bytes, which yields better robustness and language coverage but at the cost of significantly longer sequences. Charformer [6] offers a middle ground by learning subword-like units over characters through gradient-based pooling, yet it still relies on a single, fixed segmentation granularity. Our work connects to this literature by considering architectures that replace static vocabularies with learned, dynamic segmentation and examining how this change influences ICL performance.

**Hierarchical Chunking and Learned Segmentation.** Work in cognitive modeling and unsupervised sequence learning points to chunking as a central mechanism for efficient representation. The Hierarchical Chunking Model (HCM) [7] shows that models can discover variable-length units and recursively compose them into multi-level hierarchies directly from sequential data, without explicit supervision. These learned chunks align with meaningful abstractions and support transfer across tasks, suggesting that hierarchical compression provides both efficiency and interpretability. Unsupervised hierarchical RNNs [8] reach similar conclusions in language modeling, where a two-layer HRNN first forms word-to-chunk compositions and then models chunk-to-sentence structure. Chunk boundaries emerge as latent variables that adapt to downstream objectives, yet still track phrase-level structure. These studies support the idea that chunk boundaries should be learned rather than hand designed, and that multi-scale compression can reveal latent structure that fixed tokenization may obscure. H-Nets stand in this line of work, but scale the approach to large language models with end-to-end training and hybrid main networks.

**Prior Work on H-Nets.** H-Net [10] was introduced as a scalable solution for tokenizer-free modeling, demonstrating that variable-length chunking can be learned end-to-end to match BPE-based Transformer performance. Subsequent work, H-Net++ [11], extended this to morphologically rich languages, showing that learned chunks often align with linguistic morphemes and improve robustness to noise. These studies established H-Nets as valid language models; in this work, we shift the focus to their *algorithmic* capabilities, specifically evaluating how dynamic segmentation interacts

with the mechanisms of in-context learning. This analysis is complementary to prior performance-focused benchmarks.

**Efficient and Hybrid Sequence Models.** To alleviate the quadratic cost of full attention, many recent architectures pursue linear-time or hardware-friendly alternatives. Hyena [12] replaces attention with long convolutions and gated mixing, while Mamba and Mamba-2 [13, 14] use selective state space models with input-dependent dynamics to capture long-range dependencies in linear time. Gated Linear Attention (GLA) [15], DeltaNet [16], and related recurrent hybrids introduce structured updates that combine aspects of attention and RNN-style state updates, often matching Transformer-level performance with improved efficiency. RetNet [17] blends recurrence and attention to maintain a compressed state over long sequences. On the theoretical side, work on recurrent SSMs demonstrates that even shallow architectures can implement optimal Bayesian ICL for certain generative processes through convolutional smoothing and structured compression [18]. These models show that architectural inductive biases such as recurrence, smoothing, and local state updates have a profound effect on what algorithms a model can implement in context. H-Nets inhabit the same design space but impose an additional hierarchical segmentation bias, and our experiments explore how this interacts with attention and SSM-style recurrence on controlled ICL tasks.

**Mechanistic Evaluation of In-Context Learning.** A growing set of benchmarks aims to measure ICL capabilities at the level of specific algorithms rather than aggregate downstream performance. The Mechanistic Architecture Design (MAD) framework [20] provides a battery of synthetic tasks that isolate abilities such as exact recall, fuzzy recall of semantically grouped tokens, noisy recall with distractors, selective copying, compression of sequences into single latent slots, and memorization of fixed key-value maps. Zoology [19] focuses on multi-query associative recall and reveals a consistent gap between attention-free SSMs and full Transformers on MQAR-style tasks. Reg-Bench [21] studies regression and pattern-following tasks, examining how model architecture and parameterization affect higher-order generalization in context. These frameworks offer fine-grained diagnostics of what algorithms models perform when trained on next-token prediction alone. Our evaluation uses MAD as the primary testbed and interprets H-Nets’ performance through the lens of this prior work, asking how hierarchical dynamic chunking changes the balance between memory capacity, recall fidelity, and robustness across different categories of ICL tasks.

### 3 Method

Our goal is to assess how learned tokenization mechanisms in hierarchical networks like H-Nets perform on in-context learning (ICL). Specifically, we aim to answer the following research questions:

1. Does dynamic chunking help ICL relative to a flat model with the same layers and parameter budget?
2. How does hierarchical depth (1 vs 2 stages) affect H-Net’s performance across MAD tasks?

#### 3.1 Problem Setting

We consider sequence-to-sequence tasks where each example consists of an input sequence  $x = (x_1, \dots, x_L)$  and a target sequence  $y = (y_1, \dots, y_L)$  over a finite alphabet. In the original MAD-Lab benchmark, the alphabet consists of abstract tokens; here, we map each synthetic symbol to a unique character (or byte), so that all models operate directly on character-level streams. This mirrors the tokenizer-free design of H-Nets and forces architectures to discover useful span structure from raw characters.

#### 3.2 Hierarchical Network (H-Net) Architecture

Here the main network can be any standard architecture designed for operating on tokenized language—such as a Transformer or state space model (SSM). The dynamic chunking mechanism is trained end-to-end and can learn non-uniform chunk boundaries that reflect the structure of the input. The dynamic chunking introduces (i) a **router** that predicts chunk boundaries through similarity scoring, and (ii) a **smoothing component** that interpolates representations to reduce boundary uncertainty and stabilize training. These values are used in an auxiliary loss to enforce target down-sampling ratios and differentiable methods for discrete routing.

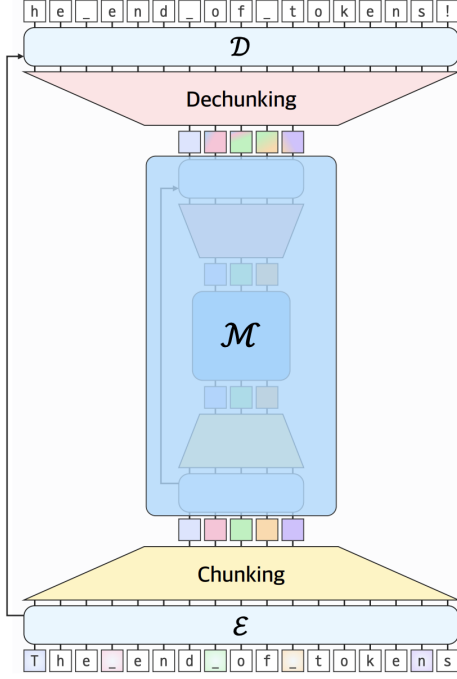


Figure 1: H-Net Network Architecture

H-Net is a hierarchical sequence model with learned dynamic chunking. H-Net adopts the hierarchical architecture resembling an autoregressive U-Net, where the model:

1. Compresses raw data into richer semantic embeddings with the **encoder network**;
2. Applies a **dynamic chunking (DC)** module to partition the encoded sequence into variable-length chunks. The dynamic chunking module interfaces between the main network and the encoder/decoder networks and learns to segment data.
3. Processes the chunk-level sequence in the **main network**;
4. Upsamples the processed chunks using the dechunking module and feeds their original resolution into the **decoder network**.

H-Net is fully end-to-end and can be iterated recursively: the main network can itself be an H-Net. Intuitively, more stages of chunking allow the model to operate on larger **multi-token groupings**; just as characters can be combined into words, words can be combined into clauses or sentences. This hierarchical aggregation aims to reduce the effective sequence length while increasing the semantic density of the units processed by the main network.

In an  $S$ -stage model, we denote components at each stage using superscripts: encoder networks as  $E^s$  and decoder networks as  $D^s$  for stages  $0 \leq s < S$ , with the main network  $M$  residing only at the final stage  $s = S$ . For example, a two-stage model contains  $E^0, E^1, M, D^1, D^0$ .

The overall pipeline can be formalized as:

$$\hat{x}^s = E^s(x^s), \quad \hat{z}^s = M(x^s), \quad \hat{z}^s = D^s(z^s). \quad (1)$$

The chunking and dechunking operations are defined as:

$$(x^{s+1}, p^s) = \text{Chunk}(\hat{x}^s), \quad (2)$$

$$z^s = \text{Dechunk}(\hat{z}^{s+1}, p^s) + \text{Linear}(\hat{x}^s). \quad (3)$$

The raw input,  $x^0 \in \mathbb{R}^{L_0 \times D_0}$ , is first encoded into features where  $L_0$  is the input sequence length and  $D_0$  is the embedding dimension. Intuitively,  $p^s \in [0, 1]^{L_s}$  represents the chunking router's confidence that each token should be passed into the next (main) stage.

At each stage  $s$ , the Chunk module aggregates these features into a shorter sequence  $x^{s+1}$  (Eq. 2). This repeats until the Main Network  $M$  processes the most compressed representation. Finally, the sequence is recursively expanded back to the original resolution via the Decoder and Dechunk module (Eq. 3), utilizing residual connections from the encoding phase.

Our aim is to access how the learned chunking mechanisms and different hierarchical depths influence H-Net's in-context learning behavior on MAD synthetic tasks. Each model is normalized to an iso-state and iso-parameter setting, allowing for fair comparisons. We experiment with the following hierarchical depths:

- **1-stage H-Net (1.1M parameters):** a single chunking and processing stage, with an encoder, main network and decoder network. For this model, we ran 2 Mamba2 layers as the

encoder (blue), 2 layers of attention and swiglu as the main network (gray), and 2 Mamba2 layers as the decoder (red).

( mamba2 mamba2 ) ( attention swiglu attention swiglu ) ( mamba2 mamba2 )

- **2-stage H-Net (1.1M parameters):** using the same layers are before, we test a two level hierarchy H-Net where the raw text are chunked twice before reaching the inner main network.

( mamba2 ( mamba2 ( attention swiglu attention swiglu ) mamba2 ) mamba2 )

- **Flatten isotropic baseline (no dynamic chunking-1.0M parameters):** this model removes the hierarchical structure entirely and processes the input as a flat sequence with no learned segmentation. It uses the same layer types and counts as 1-stage and 2-stage H-Net, and isolates the effect of learned chunking to provide a fair point of comparison.

mamba2 mamba2 attention swiglu attention swiglu mamba2 mamba2

### 3.3 Implementation

We implemented all models in the PyTorch Lightning framework to ensure reproducible and scalable training. While MAD-Lab provides common channel- and sequence-mixing primitives such as `attention` and `swiglu` layers, it does not include an H-Net implementation. Our training pipeline integrates the H-Net architecture into the MAD-Lab task suite [20], incorporating the Mamba2 layers and residual connections described in the original H-Net paper [10]. This unified setup enables consistent evaluation of different hierarchical configurations under the same training and data generation protocols.

**Architectural Justifications.** Our design choices for the H-Net components are motivated by specific inductive biases and insights from the H-Net paper:

- **Mamba-2 Encoders/Decoders:** We utilized Mamba-2 layers [14] for the encoder and decoder blocks. Unlike Transformers, which scale quadratically with sequence length, Mamba-2 offers linear-time complexity and efficient state compression. This is critical for tokenizer-free modeling, where the model must process raw byte sequences that are significantly longer than tokenized inputs. The H-Net paper ablation studies also reports that SSM-based encoders/decoders significantly outperform transformer layers [10].
- **Attention-based Main Network:** For the central main network, we employed standard Multi-Head Attention layers. Once the input is compressed into chunks by the encoder, the effective sequence length is reduced, allowing us to leverage the superior expressivity and copying capabilities of Attention [28] without incurring prohibitive computational costs. This hybrid approach aims to combine the efficiency of SSMs for low-level processing with the reasoning power of Transformers for high-level semantic manipulation [10].

The multi-head attention layers use a model width of 128, implemented as 8 heads each with a 16-dimensional subspace. The SwiGLU MLPs use an inner width of 512 (a 4× expansion from the model width) [20].

## 4 Hypothesis

**Hypothesis I: H-Nets will perform well on fuzzy recall.** In fuzzy recall, keys and values consist of multi-token spans and queries may be partially mismatched, requiring models to aggregate local information into coherent latent representations rather than rely on exact token matching. H-Nets learn variable-length chunks end-to-end, allowing the encoder to collapse semantically related spans into single latent units before main-network processing. We hypothesize that this hierarchical compression reduces sensitivity to span-boundary perturbations and enables more stable, higher-level representations of key–value structure, while flat architectures would infer span structure implicitly

through attention over raw tokens, which is more brittle under noise. We therefore expect H-Nets to exhibit greater robustness and semantic stability on fuzzy recall than flat baselines.

**Hypothesis II: 2-Stage H-Net will outperform shallower 1-Stage H-Net in MAD tasks.** We hypothesize that increasing hierarchical depth improves robustness and performance on in-context learning tasks that benefit from structured aggregation and compression, including fuzzy recall and compression. In deeper H-Nets, lower stages aggregate fine-grained token spans into intermediate representations, while higher stages operate on more abstract units, enabling the model to better capture span-level structure and salient information. By reducing the effective sequence length and entropy seen by the main network, deeper hierarchies allow later layers to implement retrieval and aggregation algorithms more reliably than shallower models.

## 5 Experiments

We now describe the datasets, training details, evaluation metrics, and baselines used in our experiments, and summarize the results. Architectural details are given in Section 3.

### 5.1 Datasets: The MAD-Lab Benchmark

To rigorously evaluate the algorithmic capabilities of H-Nets, we employ the **Mechanistic Architecture Design (MAD)** benchmark [20]. Unlike natural language benchmarks which entangle syntax, semantics, and world knowledge, MAD provides a suite of synthetic “unit tests.” Each task is designed to isolate a specific mechanistic capability required for in-context learning, such as associative recall, gating, or state compression. These aggregate MAD scores reliably predict a model’s perplexity after large-scale training [20].

**Byte-Level Adaptation.** Standard MAD tasks use a vocabulary of abstract integers (e.g.,  $V = \{0, \dots, N\}$ ). Since H-Nets are tokenizer-free architectures designed to discover structure in raw data, we map these abstract symbols to unique byte characters. This forces the model to operate on character streams, requiring the H-Net to potentially “learn” the token boundaries that are explicitly given in standard Transformer setups.

We evaluate H-Nets on the following task categories:

- **In-Context Recall (Exact Copy):** This task probes the model’s ability to implement standard associative memory (often attributed to “induction heads” in Transformers [28]). The input consists of key-value pairs followed by a query key (e.g.,  $k_1 v_1 \dots k_n v_n q_{k_i}$ ), and the model must predict the associated value  $v_i$ .
- **Noisy Recall:** This is a variant of exact recall where irrelevant “noise” tokens are interspersed between meaningful key-value pairs. This task specifically tests the architecture’s ability to *filter* information. For H-Nets, this is a crucial test of the dynamic chunking mechanism: can the encoder learn to compress noise into ignorable chunks, or does the noise disrupt the hierarchical structure?
- **Fuzzy Recall:** In this setting, keys and values are not single tokens but multi-token spans (or in our case, multi-character substrings). The query may be a slight perturbation of the original key. This tests the model’s ability to aggregate local information into a semantic representation. We hypothesize that H-Nets may excel here by learning to chunk the multi-character span into a single latent representation in the main network.
- **Selective Copying:** The model sees a stream of tokens and must copy only those that satisfy a specific criterion (e.g., distinct from a “distractor” set), preserving their relative order. This tests the implementation of *gating mechanisms* and the ability to maintain a cursor over the output sequence while skipping input elements.
- **Compression and Memorization:**
  - *Compression* requires the model to summarize a long sequence into a compact state to answer a query at the end, probing the effective memory capacity of the hierarchy.
  - *Memorization* tests the speed at which a model can internalize a fixed, static key-value mapping shared across many examples (similar to weights), rather than dynamic in-context mappings.

We generate data using the official `mad-lab` codebase, maintaining default grids for sequence length and dataset size. See Supplementary Section E.1 for detailed examples.

## 5.2 Training Details

### 5.2.1 MAD Protocol

We evaluate model performance across systematically varied levels of task difficulty, modifying only one variable at a time while keeping all others at their baseline values. Difficulty is controlled by adjusting (i) input sequence length, (ii) training dataset size, (iii) vocabulary size, and, noise.

**In-Context Recall.** We vary sequence lengths  $\{128, 256, 512, 1024\}$ , training set sizes  $\{12,800, 6,400, 3,200, 1,600, 800\}$ , and vocabulary sizes (split evenly into keys and values)  $\{16, 32, 64, 128\}$ .

**Noisy In-Context Recall.** In addition to the above variations, we vary the noise-token share in the input:  $\{20\%, 40\%, 60\%, 80\%\}$ .

**Selective Copying.** We evaluate sequence lengths  $\{256, 512, 1024\}$ , training set sizes  $\{12,800, 6,400, 3,200, 1,600, 800\}$ , vocabulary sizes  $\{16, 32, 64, 128\}$ , and the number of tokens to copy  $\{16, 32, 64, 96\}$ .

**Compression.** We vary sequence lengths  $\{32, 64, 128, 256\}$ , vocabulary sizes  $\{16, 32, 64, 128\}$ , and training set sizes  $\{12,800, 6,400, 3,200, 1,600, 800\}$ .

**Memorization.** We vary vocabulary sizes  $\{256, 512, 1,024, 2,048, 4,096, 8,192\}$ , with the training set fixed at 256 examples and input length fixed at 32, thereby controlling the average number of occurrences of each fact  $\{32, 16, 8, 4, 2, 1\}$ .

### 5.2.2 Hyperparameters and Optimization

We train the flat H-Net baseline, the 1-stage H-Net, and the 2-stage H-Net across all MAD tasks. All models use a batch size of 128 and are trained for 50 epochs with the AdamW optimizer (momentum parameters  $\beta_1 = 0.9, \beta_2 = 0.98$ ), together with early stopping to prevent overfitting. We employ a cosine learning-rate schedule and use 8 layers for every architecture variant. Each model-task pair is evaluated on 1,280 held-out/evaluation samples.

Following the MAD benchmark protocol [20], we sweep over a  $3 \times 2$  grid of learning-rate and weight-decay values, using learning rates  $\{1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}\}$  and weight decays  $\{0.0, 0.1\}$ . These hyperparameters correspond to the default configuration used in MAD as described in the Appendix B.

## 5.3 Training objective

All models are trained with a standard autoregressive cross-entropy loss. Given an input sequence  $x = (x_1, \dots, x_L)$  and target sequence  $y = (y_1, \dots, y_L)$ , we minimize

$$\mathcal{L}(\theta) = -\frac{1}{L} \sum_{t=1}^L \log p_{\theta}(y_t | x_{1:t}),$$

where  $p_{\theta}$  is the model’s predictive distribution over the next character.

## 5.4 Metrics

On the MAD-Lab tasks, our primary metric is token-level accuracy on a held-out evaluation set. Given predictions  $\hat{y}_t^{(i)}$  and targets  $y_t^{(i)}$  for  $N_{\text{eval}}$  sequences of length  $L$ , we compute

$$\text{Acc} = \frac{1}{N_{\text{eval}} L} \sum_{i=1}^{N_{\text{eval}}} \sum_{t=1}^L \mathbf{1}\{\hat{y}_t^{(i)} = y_t^{(i)}\}.$$

For each task we report accuracies averaged across MAD-Lab difficulty levels (per-task mean accuracy), and we define an aggregate “MAD score” as the mean of per-task mean accuracies across all tasks. We also track evaluation cross-entropy, but use accuracy as the main metric.

## 5.5 Results

**MAD-Lab.** Our main quantitative results on MAD-Lab are summarized in a single table of per-task accuracies and aggregate MAD scores table:

| Model           | Compress | Fuzzy Recall | In-Context Recall | Memorize | Noisy Recall | Selective Copy | MAD score   |
|-----------------|----------|--------------|-------------------|----------|--------------|----------------|-------------|
| H-Net flattened | 100.0    | 31.2         | 99.3              | 84.6     | 96.6         | 97.7           | <b>84.9</b> |
| H-Net 1-Stage   | 100.0    | 21.4         | 99.82             | 82.8     | 92.9         | 91.6           | <b>81.4</b> |
| H-Net 2-stage   | 100.0    | 25.7         | 99.87             | 80.4     | 99.7         | 91.3           | <b>82.8</b> |

Table 1: Comparing H-Net structure results across MAD tasks

Each task score is the average performance across all the difficulty levels. The MAD scores are computed as the average performance across all individual in-context tasks. We observe that the flattened H-Net, which does not employ any learned hierarchical chunking, performs the best overall across the full MAD suite.

**Compression.** All H-Net variants achieve 100% accuracy on the compression task. Even in the hardest setting (sequence length 256, vocabulary size 128), the amount of information that must be retained is small relative to the  $\sim 1.1\text{M}$  parameters in our flattened, 1-stage, and 2-stage H-Nets, and the Mamba-2 encoder/decoder layers bring a strong inductive bias toward long-range compression [10]. Thus our H-Net configurations are overparameterized for this task and make it essentially trivial, so compression provides little signal to distinguish the H-Net variants.

**Fuzzy Recall.** On fuzzy recall, the flattened baseline performs best, while both hierarchical H-Net variants lag behind. Because keys and values are represented as multi-token spans, the model must infer span boundaries rather than match single tokens, making the task substantially more challenging. Although the hierarchical models underperform relative to the flat baseline, the 2-stage H-Net consistently outperforms the 1-stage variant, suggesting that additional chunking depth helps the model capture span-level structure, albeit imperfectly. We also observed slower convergence for H-Nets on the hardest fuzzy recall settings, indicating that additional training epochs may reduce this gap. While this contradicts our initial hypothesis that hierarchical chunking would outperform flat processing on fuzzy recall, the consistent improvement from 1-stage to 2-stage H-Nets suggests that increasing hierarchical depth may benefit fuzzy recall, motivating further exploration of deeper hierarchies.

**In-Context Recall.** On standard in-context recall, H-Net performs very strongly. Our Flattened, 1-stage, and 2-stage H-Net variants all reach near perfect accuracy on the byte level MAD adaptation. We also see a clear trend where performance improves as we increase the number of stages in the hierarchy. This fits the original motivation for H-Nets: in-context recall is exactly the kind of task where hierarchical segmentation should help the model structure and retrieve information in the prompt.

**Memorization.** On the memorization task, the flattened baseline performs best, followed by the 1-stage H-Net, with the 2-stage H-Net performing worst. Here the model must learn a fixed key value mapping from the training data that remains constant across samples, so there is no input dependent structure to exploit at test time and simpler architectures are favored. A flat model can directly internalize the static dictionary into its parameters, while multi stage processing offers little benefit and can even hinder optimization by spreading representations of these static mappings across stages. The monotonic ordering (flat > 1-stage > 2-stage) supports the idea that hierarchical chunking is most useful when there is structure to infer in context rather than a global lookup table to memorize.

**Noisy Recall.** On noisy recall, the 2-stage H-Net achieves almost perfect accuracy (99.7%), outperforming both the flattened baseline (96.6%) and the 1-stage H-Net (92.9%). In Figure 4 the 2-stage curve climbs slightly more slowly at the beginning, then continues to improve and plateaus very close to 1.0. We speculate that the extra hierarchy may allow the model to act as a learned

denoiser, potentially collapsing long runs of distractor tokens into a small number of low salience chunks. While we lack direct visualization of the learned chunks to confirm this mechanism, the performance gap suggests the deeper hierarchy effectively filters noise that disrupts shallower models.

**Selective Copy.** For selective copy, all three architectures reach high average accuracy in the MAD grid (flattened H-Net at 97.7%, 1-stage and 2-stage H-Nets around 91%), but the learning dynamics are less stable. In the hardest setting with a fixed learning rate (Figure 5), the flattened model learns the copying algorithm very quickly, jumping to near perfect test accuracy within the first ten epochs, then experiencing several large drops before recovering. The hierarchical variants improve more gradually and, under this single hyperparameter choice, still underfit by epoch 50. This instability matches the structure of the task: only the tokens that must be copied contribute to the gradient, and the objective is very sensitive to small alignment errors, so minor changes in attention patterns or chunk boundaries can cause many copied tokens to shift and produce sharp swings in sequence level accuracy even when token level loss changes only modestly. The fact that the full learning rate and weight decay sweeps recover strong final performance for all three models suggests these oscillations are mainly an optimization artifact rather than a fundamental limitation of the hierarchical architectures on selective copy.

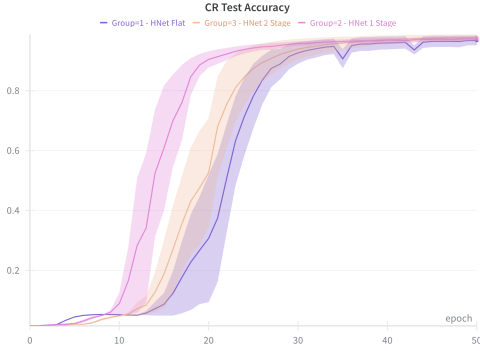


Figure 2: In-Context Recall

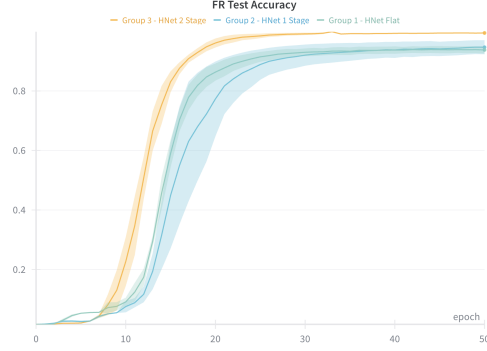


Figure 3: Fuzzy Recall

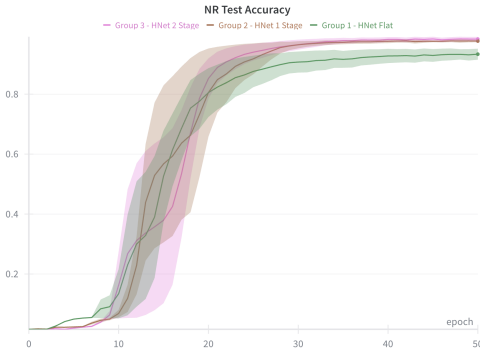


Figure 4: Noisy Recall

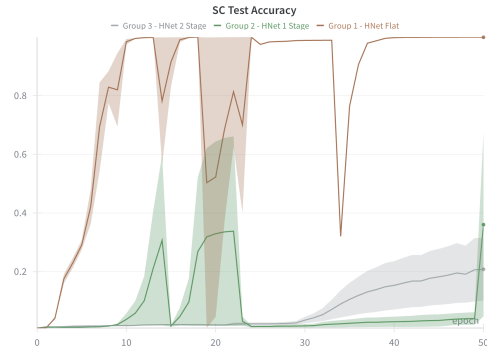


Figure 5: Selective Copy

Figure 6: Test Accuracy vs. Epoch for In-Context Recall, Fuzzy Recall, Noisy Recall, and Selective Copy.

## 5.6 Test Curves

It is important to distinguish the experimental setup between the aggregate scores and the learning curves. The benchmark results in Table 2 were obtained using the full  $3 \times 2$  hyperparameter sweep to

find the optimal configuration. In contrast, the per-task learning curves reported here (Figure 6) use a **fixed learning rate** of  $5 \times 10^{-4}$  and zero weight decay to illustrate training dynamics. These were the default optimal hyperparameters described in the MAD paper for testing without hyperparameter sweeps. Consequently, the performance in Figure 6 may differ from the peak performance reported in Table 2.

### 5.7 Model Performance on Individual MAD Tasks

Across all MAD tasks, increasing the number of training examples consistently improves performance, leading to higher accuracy and lower perplexity, with gains saturating at larger scales. Increasing vocabulary size makes tasks harder across the board, generally decreasing accuracy and increasing perplexity due to higher input entropy. Changes in sequence length have a weaker and more task-dependent effect, with longer sequences slightly increasing perplexity but often leaving accuracy unchanged once it has saturated. Architectural differences matter most on harder tasks, while on easier tasks all models perform similarly; full task-level results are provided in Appendix Section C.

## 6 Conclusion

In this work, we investigated the in-context learning capabilities of Hierarchical Networks (H-Nets), a tokenizer-free architecture that learns to segment sequences dynamically. We integrated the H-Net architecture into the MAD-Lab framework to evaluate its performance on synthetic reasoning tasks and conducted a focused ablation study on a subset of Wikipedia to assess its behavior on natural language.

Our experiments revealed a distinct trade-off between strict character-level accuracy and semantic content retrieval in compression and fuzzy recall. In our ablation study, while the baseline character-level Transformer achieved slightly higher raw accuracy, the H-Net demonstrated significantly higher Jaccard similarity (0.3979 vs. 0.1480). This result suggests that the dynamic chunking mechanism successfully captures the "bag of information"—the essential semantic content of the context—even when it does not perfectly reproduce the exact character sequence. This supports the hypothesis that learned, hierarchical segmentation provides a beneficial inductive bias for capturing high-level structure, potentially offering greater robustness than fixed tokenization in low-resource or noisy settings.

Future work will focus on scaling H-Nets to deeper hierarchies and larger datasets to determine if this semantic robustness translates to superior few-shot performance on complex downstream tasks. We also aim to further explore the synergy between dynamic chunking and efficient state-space backbones like Mamba.

## Limitations & Acknowledgements

While our results highlight the potential of H-Nets for ICL, there are limitations to our experimental scope due to computational constraints. First, we were unable to train tokenizer-free baselines such as ByT5 or Charformer, or a byte-level Transformer, which would provide more direct comparisons than the token-based reference models. Second, due to compute limits, we did not perform extensive error bar analysis on the aggregate MAD scores (Table 2); however, individual training runs in Appendix C illustrate variance. Regarding compression ratios, we deferred to the default settings specified in the MAD-Lab reference paper. Future work should address these gaps by scaling up the baseline comparisons and visualizing the learned latent segmentations.

We would like to thank the course staff of CS 182 at UC Berkeley for their guidance and support throughout this project, as well as the peer reviews articulating the need for more results and training runs. We also acknowledge the use of approximately 1300 units of compute resources from google colab pro (valued at more than \$100) utilized for training our models and running ablation studies.

## References

- [1] Tom B. Brown *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901, 2020.
- [2] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the ACL*, pages 1715–1725, 2016.
- [3] Alex Wang *et al.* Tokenization Matters! Challenging Their Tokenization. *arXiv preprint arXiv:2406.11687*, 2024.
- [4] Felix Hieber *et al.* On the limitations of fixed tokenization in large language models. *arXiv preprint*, 2024.
- [5] Xue *et al.* ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.
- [6] Yi Tay *et al.* Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations (ICLR)*, 2022.
- [7] Zhoutong Wu *et al.* Learning structure from the ground up: Hierarchical representation learning by chunking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [8] Zhoutong Wu *et al.* Unsupervised chunking with hierarchical RNNs. *arXiv preprint*, 2023.
- [9] Junyoung Chung, Sungjin Ahn, and Yoshua Bengio. Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [10] Sungwon Hwang *et al.* H-Net: Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*, 2025.
- [11] Maryam Zakershahrok and Seyed Sam Ghodratinama. H-Net++: Hierarchical dynamic chunking for tokenizer-free language modeling in morphologically-rich languages. *arXiv preprint arXiv:2508.05628*, 2025.
- [12] Michael Poli *et al.* Hyena hierarchy: Towards larger convolutional language models. *arXiv preprint arXiv:2302.10866*, 2023.
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [14] Tri Dao and Albert Gu. Transformers are SSMs: Structured state-space duality (Mamba-2). *arXiv preprint arXiv:2405.21060*, 2024.
- [15] Linyi Yang *et al.* Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2306.14795*, 2023.
- [16] Linyi Yang *et al.* Parallelizing linear transformers with the delta rule (DeltaNet). *arXiv preprint*, 2025.
- [17] Zhiliang Sun *et al.* Retentive network: A successor to transformers for LLMs. *arXiv preprint arXiv:2307.08621*, 2023.
- [18] Julia Bondaschi *et al.* From Markov to Laplace: How Mamba in-context learns Markov chains. *arXiv preprint*, 2025.
- [19] Kanishk Arora *et al.* Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- [20] Ahmet Üstün Akyürek *et al.* Mechanistic architecture design (MAD): Synthetic unit tests for in-context learning. *arXiv preprint arXiv:2403.17844*, 2024.
- [21] Ahmet Üstün Akyürek *et al.* RegBench: Benchmarking in-context learning with regular languages. *arXiv preprint arXiv:2401.12973*, 2024.
- [22] Jason Wei *et al.* Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [23] Jason Wei *et al.* Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846*, 2023.
- [24] Sewon Min *et al.* Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of NAACL*, 2022.

- [25] Zihang Dai *et al.* Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of ACL*, pages 2978–2988, 2019.
- [26] Jack W. Rae *et al.* Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.
- [27] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.
- [28] Catherine Olsson *et al.* In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.

## A Addressing Peer Reviews

Below we summarize the main feedback received during the poster session and formal peer review, along with the corresponding changes made to the paper.

### A.1 Feedback from the Poster Session

- **Choice of MAD synthetic dataset.** Several peers asked why we chose the MAD synthetic benchmark to evaluate H-Net’s in-context learning abilities.
  - *Response:* We chose MAD because it provides controlled, mechanistic tasks (e.g., fuzzy recall and compression) that directly probe in-context learning while minimizing confounding semantic priors from natural language data. We clarified this motivation and added a brief comparison to natural-language benchmarks in Section ??.
- **Clarity of H-Net architecture and formulas.** Some attendees requested a more explicit mathematical description of the H-Net architecture.
  - *Response:* We expanded the architectural description in Section 3, adding explicit formulas for the encoder, hierarchical chunking mechanism, and main network, along with an additional schematic diagram.

### A.2 Feedback from Peer Reviewers

- **Add a Hypothesis Section.** Two reviewers mentioned that while we talked about our hypothesis throughout our paper, it would improve readability to have an explicit section for our hypothesis section.
  - *Response:* We added a hypothesis section to the paper in Section 4.
- **One reviewer mentioned that the plots do not have error bars.**
  - *Response:* The reviewer is likely referring to the plots showing model performance across individual MAD tasks in Section C. These figures illustrate model accuracy as a function of task difficulty, including variations in vocabulary size, sequence length, and number of training examples. We follow the same plotting convention as the original MAD benchmark paper and they do not report seeds for these difficulty sweep graphs [20]. Difficulty sweeps follow MAD’s single-seed protocol, while learning curves are averaged over three runs. For our main evaluation plots reporting test accuracy, we include error bars corresponding to the standard deviation across three independent runs. Each run uses a newly generated dataset and independently trained model. We show graphs for the test accuracy, test loss, training accuracy, training loss with error bars across In-Context recall, fuzzy recall, noisy recall, and selective copy. Error bars are not shown for the memorization and compression tasks because all models converge to near-perfect accuracy within a single epoch, resulting in negligible variance.
- **One reviewer mentioned that “a clearer diagram showing how data moves through the encoder, chunking module, main network, and decoder would help,” while another reviewer noted that they “especially liked that they included a model image, it made it a lot easier to visualize their flow.”**
  - *Response:* We appreciate both comments. The manuscript already includes a diagram illustrating the data flow through the encoder, dynamic chunking module, main network, and decoder, which one reviewer specifically highlighted as helpful for visualizing the architecture. In addition, we expanded the surrounding text to provide more detailed explanations of the architectural components and the chunking and dechunking equations. For this reason, we believe the current diagram accurately conveys the H-Net architecture.
- **One reviewer suggested adding a hyperparameters table.**
  - *Response:* While all hyperparameters were previously listed in paragraph form, we added an explicit hyperparameters table to improve readability in section (Section B) that summarizes all training settings and their justifications.

- **Tokenization discussion overlap.** The reviewer noted that the discussion of tokenization limitations on pages 1 and 2 contained overlap and could be tightened.
  - *Response:* We revised the text in the introduction and background sections to reduce redundancy and tighten the arguments regarding the limitations of fixed tokenization.
- **Clarity of “higher-order meanings.”** The reviewer pointed out that “higher-order meanings” in Section 3.2 was defined only through analogy.
  - *Response:* We updated the terminology to use “multi-token groupings” (or similar precise phrasing) to provide a concrete definition rather than relying on analogy.
- **Baseline comparisons in Table 1.** The reviewer raised concerns that the comparison in Table 1 was not “apples-to-apples” due to differences in layer count and training epochs, making the claim that H-Net “outperforms” misleading.
  - *Response:* We moved Table 1 to the Appendix to reflect that these are not direct comparisons. We also removed the textual claims that H-Net “outperforms” the listed architectures given the different configurations.
- **Error bars and significance tests in Table 2.** The reviewer noted that Table 2 lacked error bars, making it hard to interpret stability compared to the learning curves in Appendix C.
  - *Response:* We clarified in the text that Table 2 represents benchmark results averaged across hyperparameter sweeps where error bars were computationally infeasible. To address the concern, we referenced the error bars provided for individual training runs in the figures to demonstrate stability and explicitly noted the compute limitations regarding the full benchmark.
- **Missing baselines (ByT5, Charformer, Byte-level Transformer).** The reviewer identified ByT5, Charformer, and byte-level Transformers as natural baselines that were missing from the experiments.
  - *Response:* We added a discussion in the “Limitations and Future Directions” section acknowledging these as relevant baselines and explicitly stating that they were omitted due to computational constraints.
- **Experimental inconsistencies and missing details.** The reviewer noted inconsistencies between the hyperparameters in Section 4.7 vs. Table 2, and missing details regarding early stopping and compression ratios.
  - *Response:* We clarified the distinction between the table results (full hyperparameter sweeps) and the learning curves (single best parameter set). We also explicitly added the early stopping criteria (test accuracy > 99.99%) and deferred to the MAD paper reference for specific details on target compression ratios.
- **Speculation on “Learned Denoiser.”** The reviewer argued that calling the chunking mechanism a “learned denoiser” was speculation without visualization evidence.
  - *Response:* We revised the text to explicitly clarify that the “learned denoiser” interpretation is a hypothesis/speculation on our part, rather than a proven mechanism, as we did not include chunk visualizations. In addition, due to the limited time between receiving peer-review feedback and the project deadline, we were unable to visualize the learned chunks produced by the H-Net models. We view this analysis as a natural extension of the project, but we are still able to clearly communicate the main findings of our study without these visualizations.
- **Concerns about the Wikipedia ablation and missing baselines.** The reviewer noted that the Wikipedia ablation is too limited to support strong conclusions, that there is no byte-level Transformer baseline on MAD tasks, and that the target compression ratio is not specified for the main experiments.
  - *Response:* We emphasize that the Wikipedia experiment is intended purely as an ablation study to probe H-Net behavior beyond synthetic MAD tasks, rather than as a core empirical result. For this reason, these results are not discussed in the main results section and are instead placed in the supplementary material, where they serve as a qualitative analysis for readers interested in training H-Nets on real-world data. Our primary conclusions are drawn exclusively from controlled comparisons on the MAD benchmark, where all models operate under identical settings. We agree that a

byte-level Transformer baseline on MAD tasks would be a valuable comparison and view this as an important direction for future work. Finally, we clarify that the target compression ratio is fixed according to the default H-Net configuration and is now explicitly specified in the supplementary section describing the ablation setup.

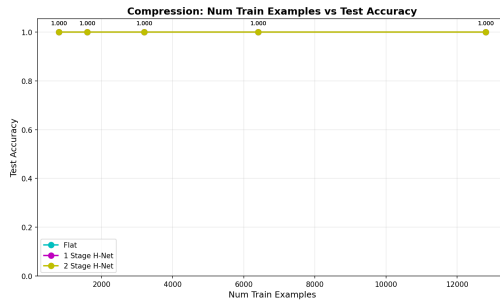
## B Hyperparameters

Table 2: Training hyperparameters and justifications for all H-Net variants on MAD tasks.

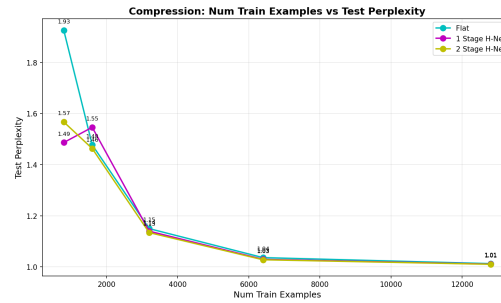
| Hyperparameter         | Value                                                      | Justification                                                                |
|------------------------|------------------------------------------------------------|------------------------------------------------------------------------------|
| Models                 | Flattened, 1-stage, 2-stage H-Net                          | Enables controlled comparison of hierarchical depth under identical settings |
| Batch size             | 128                                                        | Matches the default MAD benchmark configuration                              |
| Training epochs        | 50 (early stopping)                                        | Sufficient for convergence on MAD tasks while preventing overfitting         |
| Optimizer              | AdamW                                                      | Default optimizer used in the MAD benchmark                                  |
| AdamW $\beta_1$        | 0.9                                                        | Momentum parameter specified by the MAD protocol                             |
| AdamW $\beta_2$        | 0.98                                                       | Momentum parameter specified by the MAD protocol                             |
| Learning rate schedule | Cosine decay                                               | Smoothly anneals learning rate to improve convergence                        |
| Learning rate sweep    | $\{1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}\}$ | Follows the MAD protocol to account for task-dependent sensitivity           |
| Weight decay sweep     | $\{0.0, 0.1\}$                                             | Regularization values specified by the MAD benchmark                         |
| Hyperparameter grid    | $3 \times 2$                                               | Ensures fair model selection consistent with MAD evaluation                  |
| Number of layers       | 8                                                          | Keeps model depth fixed across all architectural variants                    |
| Evaluation samples     | 1,280 per task                                             | Matches MAD benchmark evaluation size for statistical consistency            |

## C Model Performance on Individual MAD Tasks

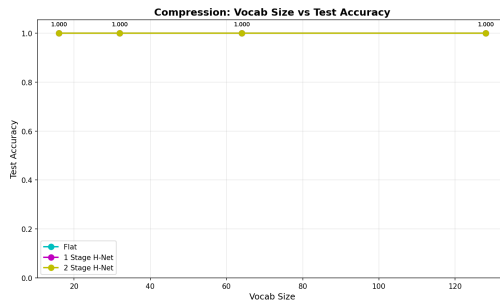
### C.0.1 Compression



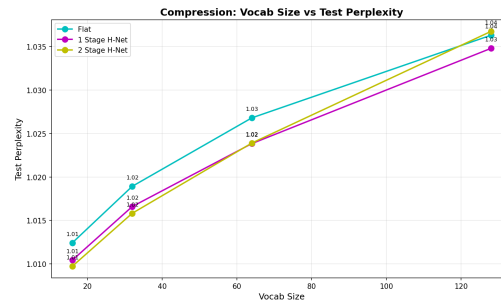
Accuracy vs. Number of Train Examples



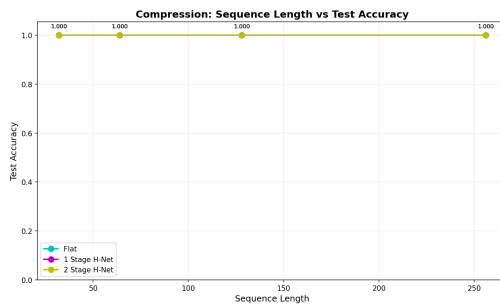
Perplexity vs. Number of Train Examples



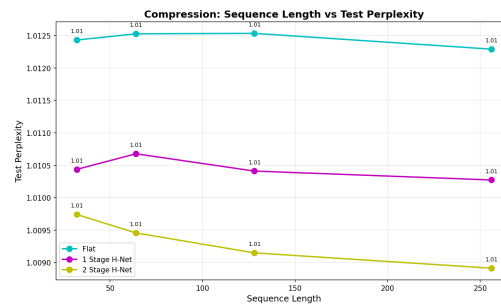
Accuracy vs. Vocabulary Size



Perplexity vs. Vocabulary Size



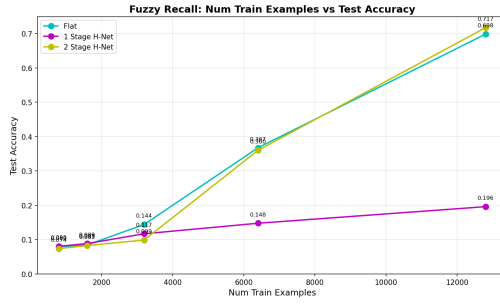
Accuracy vs. Sequence Length



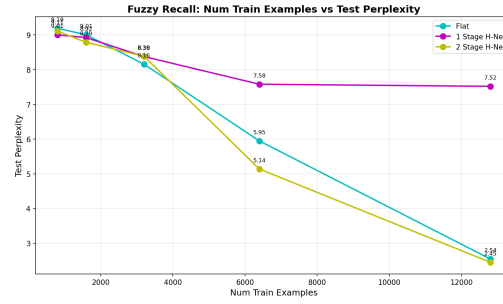
Perplexity vs. Sequence Length

Figure 7: Performance of all 3 models across varying training examples, vocabulary sizes, and sequence lengths on the Compression task. Accuracy and loss plots are shown side-by-side for each variable.

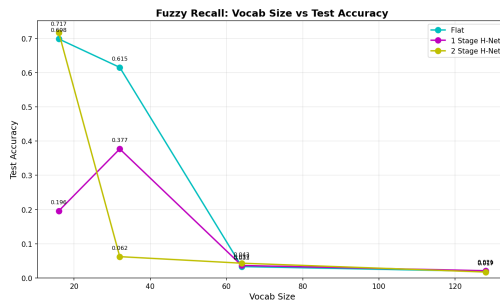
## C.0.2 Fuzzy In-Context



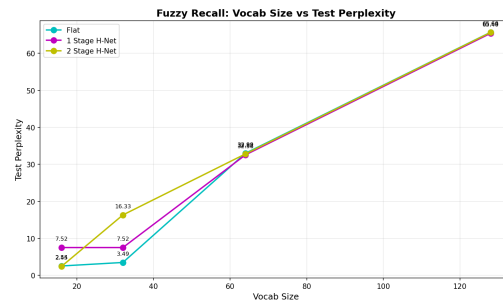
Accuracy vs. Number of Train Examples



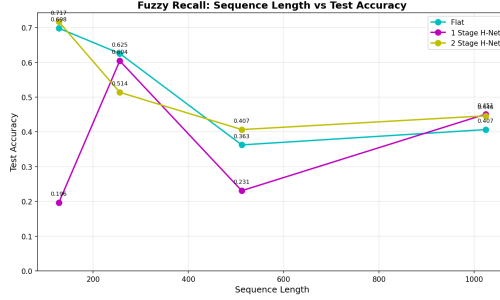
Perplexity vs. Number of Train Examples



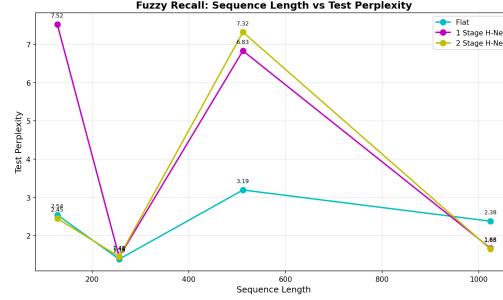
Accuracy vs. Vocabulary Size



Perplexity vs. Vocabulary Size



Accuracy vs. Sequence Length



Perplexity vs. Sequence Length

Figure 8: Performance of all 3 models across varying training examples, vocabulary sizes, and sequence lengths on the fuzzy-in-context task. Accuracy and loss plots are shown side-by-side for each variable.

### C.0.3 Memorization

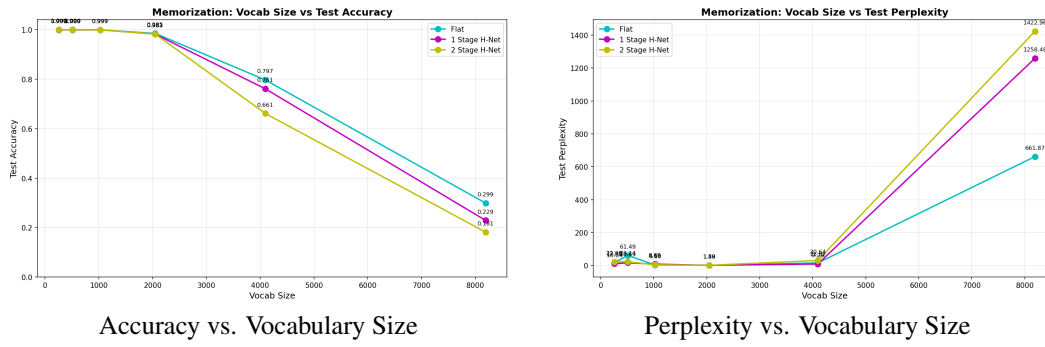
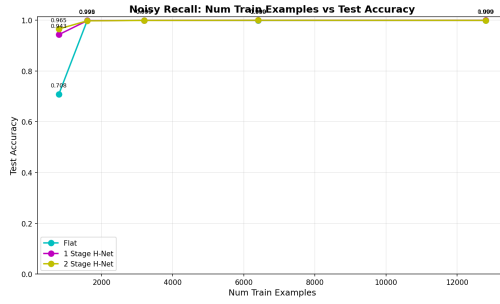
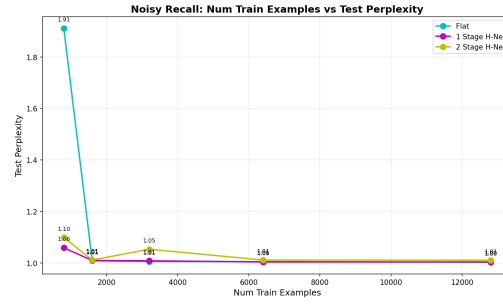


Figure 9: Performance of all 3 models across varying vocabulary sizes on the Memorization task. Accuracy and loss plots are shown side-by-side for each variable.

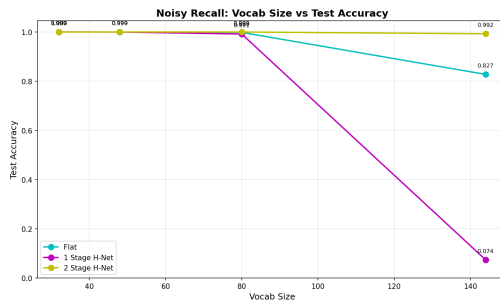
## C.0.4 Noisy In-Context



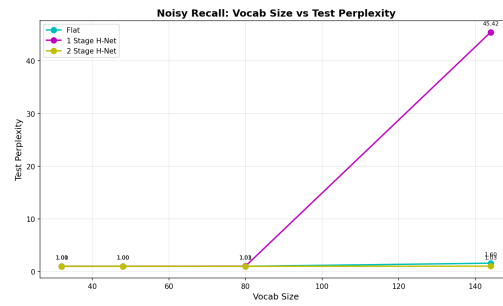
Accuracy vs. Number of Train Examples



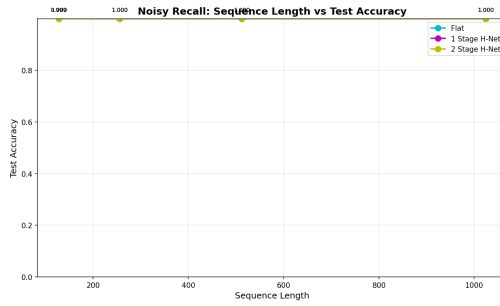
Perplexity vs. Number of Train Examples



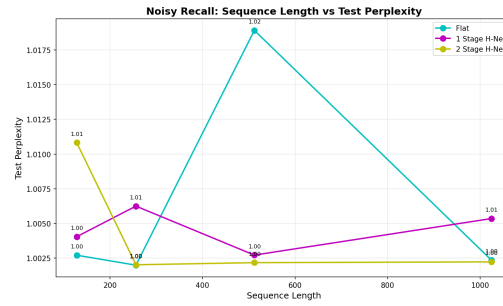
Accuracy vs. Vocabulary Size



Perplexity vs. Vocabulary Size



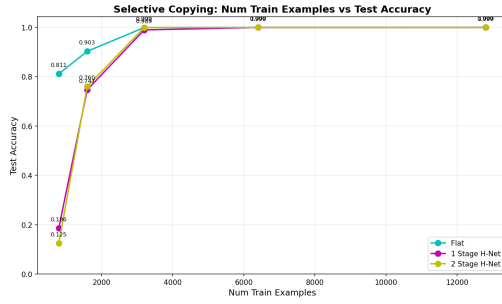
Accuracy vs. Sequence Length



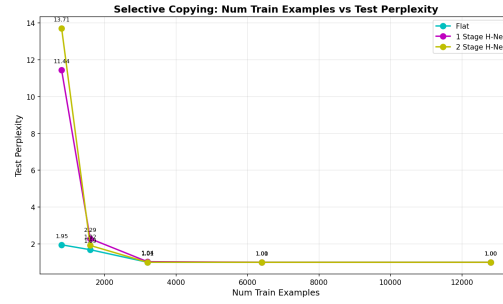
Perplexity vs. Sequence Length

Figure 10: Performance of all 3 models across varying training examples, vocabulary sizes, and sequence lengths on the noisy-in-context task. Accuracy and loss plots are shown side-by-side for each variable.

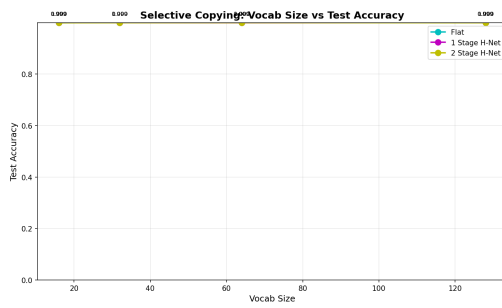
## C.0.5 Selective Copying



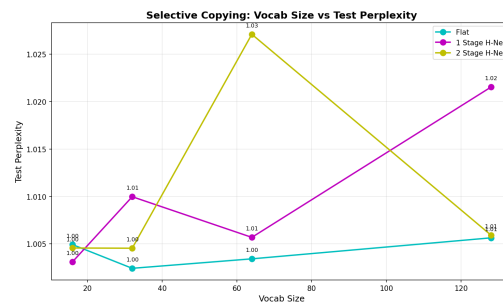
Accuracy vs. Number of Train Examples



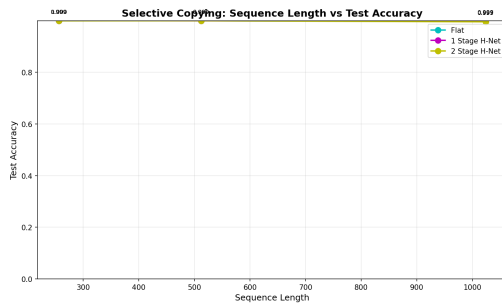
Perplexity vs. Number of Train Examples



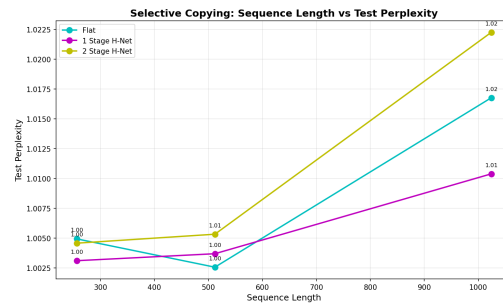
Accuracy vs. Vocabulary Size



Perplexity vs. Vocabulary Size



Accuracy vs. Sequence Length



Perplexity vs. Sequence Length

Figure 11: Performance of all 3 models across varying training examples, vocabulary sizes, and sequence lengths on the selective copy task. Accuracy and loss plots are shown side-by-side for each variable.

## D Training and Evaluation Curves

### D.1 In-Context Recall

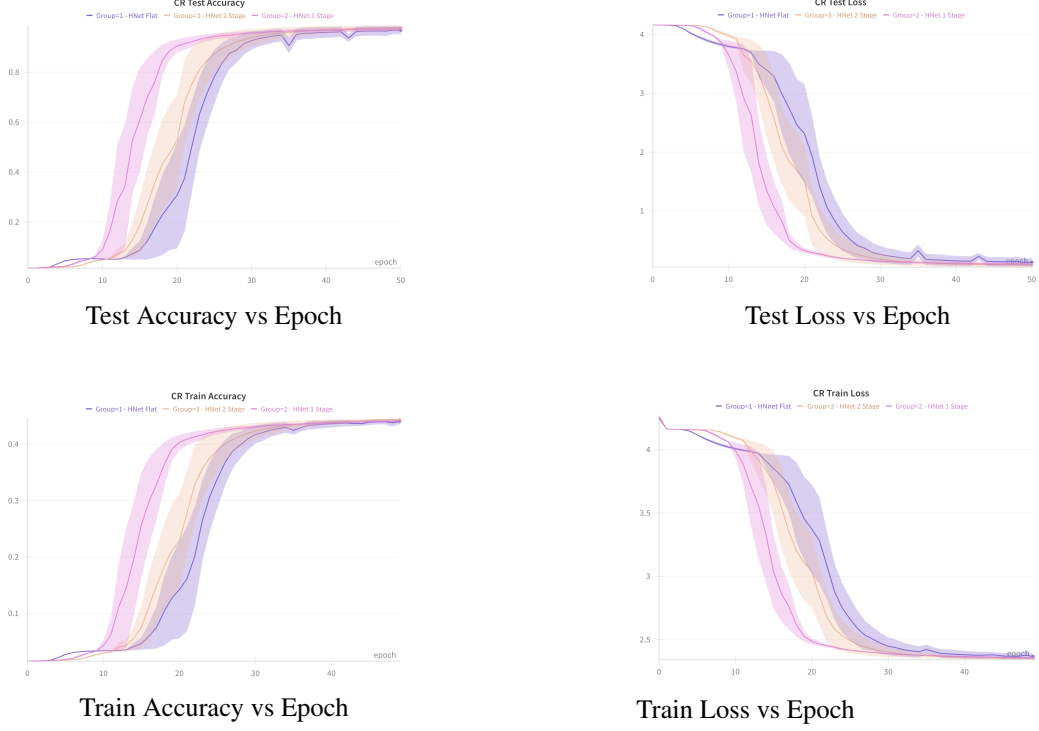


Figure 12: Validation and test accuracy/loss curves for the in-context recall task, showing model performance across training. Each model is trained 3 times, each time generates new synthetic results. For these curves, we evaluate the hardest MAD configuration for in-context recall: vocabulary size 128, sequence length 1024, and the default 128,000 training examples.

## D.2 Fuzzy Recall

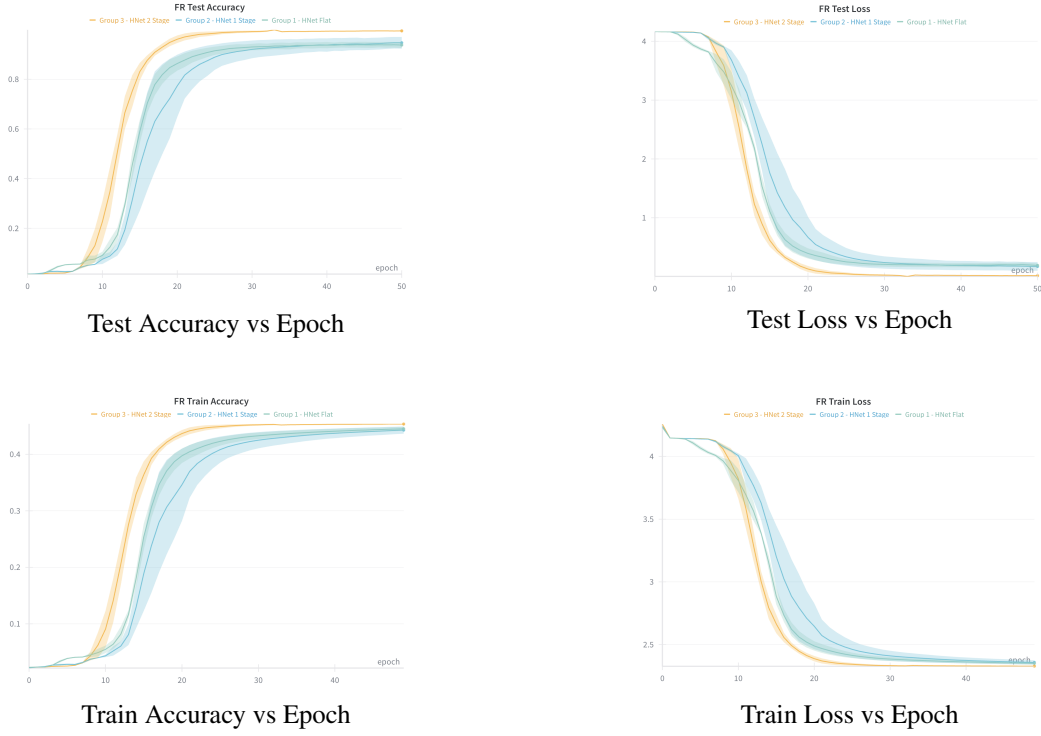


Figure 13: Validation and test accuracy/loss curves for the fuzzy recall task, showing model performance across training. Each model is trained 3 times, each time generates new synthetic results. For these curves, we evaluate the hardest MAD configuration for fuzzy recall: vocabulary size 128, sequence length 1024, and the default 128,000 training examples.

### D.3 Noisy Recall

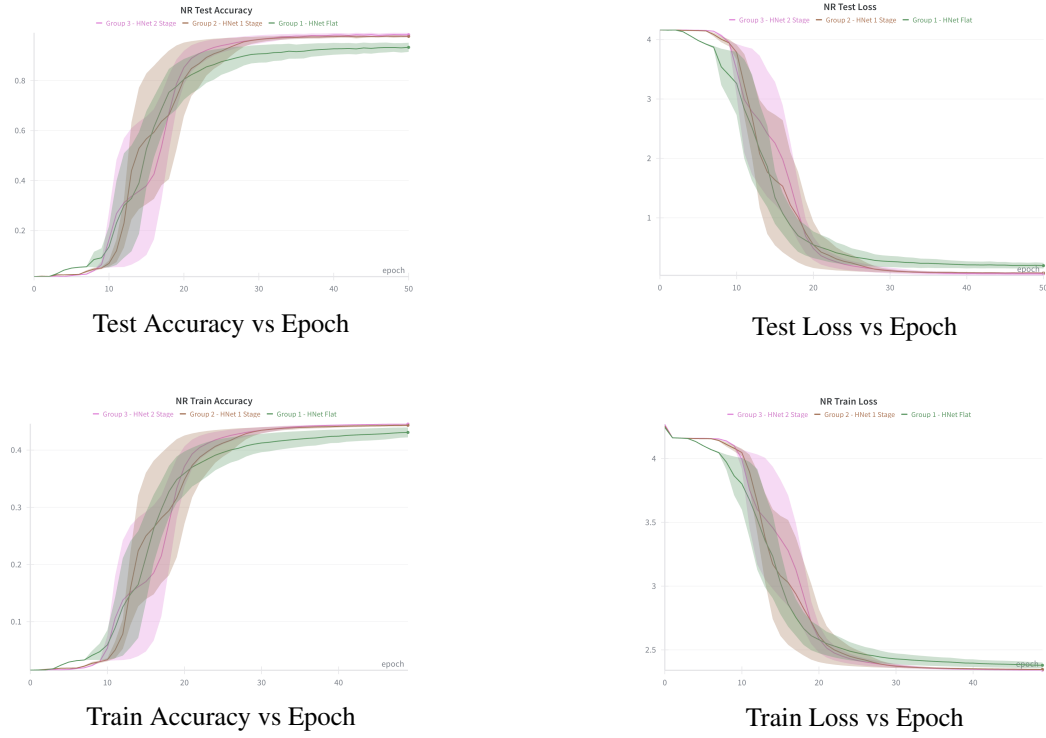


Figure 14: Validation and test accuracy/loss curves for the noisy recall task, showing model performance across training. Each model is trained 3 times, with each run generating new synthetic data. For these curves, we evaluate the hardest MAD configuration for noisy recall: noise vocabulary size 16, total vocabulary size 144, sequence length 1024, noise fraction  $\text{frac\_noise} = 0.8$ , and the default 128,000 training examples.

#### D.4 Selective Copy

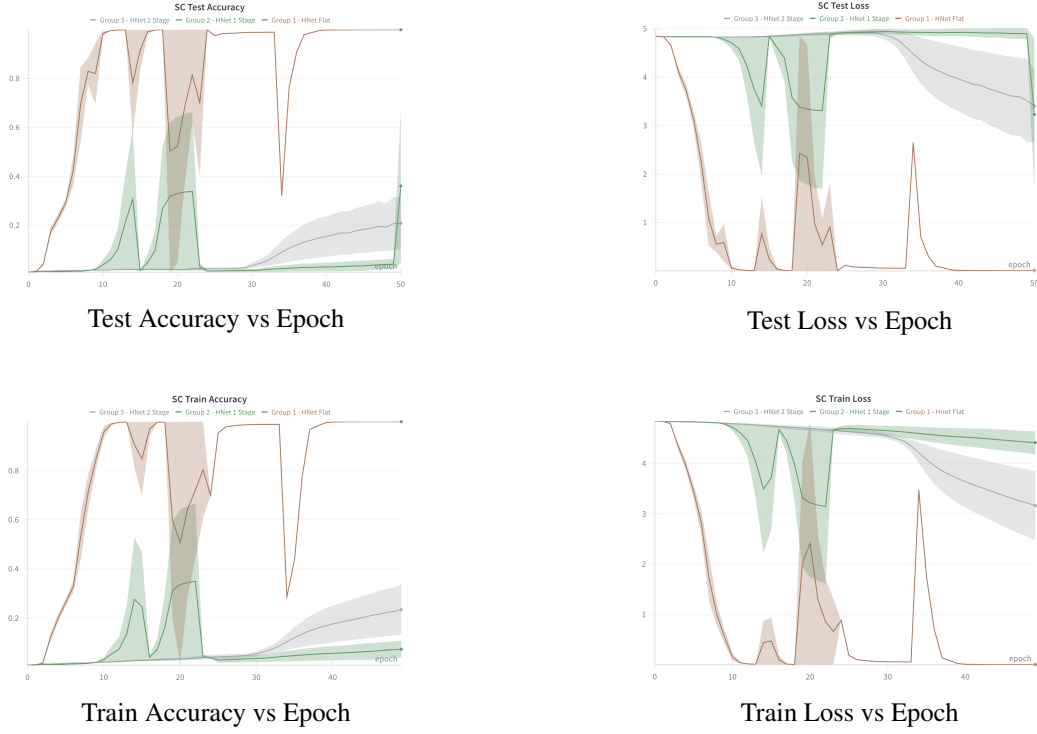


Figure 15: Validation and test accuracy/loss curves for the selective copy task, showing model performance across training. Each model is trained 3 times, each time generates new synthetic results. For these curves, we evaluate the hardest MAD configuration for recall: vocabulary size 128, sequence length 1024, number of tokens to copy 96, and the default 128,000 training examples.

## E Mechanistic Architecture Design

### E.1 Task Examples from MAD paper

#### E.1.1 In-Context Recall

The in-context recall task is comprised of sequences of key-value pairs (with separate vocabularies for keys and values). Models are tasked with predicting all values for those keys that were already presented in the sequence:

input example: a b d e f g — a b f g

In this example, keys are drawn from the vocabulary {a, d, f} and values from the {b, e, g} vocabulary. Importantly, the mapping from keys to values is randomly shuffled between sequences. Models are tasked with autoregressively predicting all underlined value in this example.

In the baseline setting of this task, we use a vocabulary of 16 tokens and 12,800 training sequences with a length of 128 tokens. The vocabulary is equally divided into keys and values.

#### E.1.2 Fuzzy In-Context Recall

The fuzzy in-context recall tasks adapts the in-context recall task by representing keys and values by a variable number of adjacent tokens:

input example: (a d) (b) (d a f) (e g) | (d a f) (e g)

In this example, keys are drawn from the vocabulary {a, d, f} and values are drawn from the vocabulary {b, e, g}. We use brackets for illustrative purposes to indicate adjacent tokens that together represent a key or value but they are not part of the actual input to the model. In sequential order, the presented keys are 'a d' and 'd a f', with associated values 'b' and 'e g'. For each sequence, keys and values are randomly drawn from the key and value dictionaries, with randomly drawn lengths (ranging from 1 to 3 tokens in our analyses). We always evaluate with keys of length 3 (the longest length used in our analyses), to disambiguate whenever a key token appears in two keys of different values. We pad sequences with a separate pad token if necessary to ensure that all sequences of a dataset are of the exact same length. As for the in-context recall task, models are tasked with autoregressively predicting all underlined values in this example.

In the baseline setting of this task, we use a vocabulary of 16 tokens and 12,800 training sequences with a length of 128 tokens. The vocabulary is equally divided into key and value tokens.

#### E.1.3 Noisy In-Context Recall

The noisy in-context recall task represents another variation of in-context recall, in which noise tokens, from a separate vocabulary, are randomly inserted into the input sequences:

input example: a b h d e f g | i a b f g

In this example, keys and values are respectively drawn from the vocabularies {a, d, f} and {b, e, g}, while noise is drawn from the vocabulary {h, i}. As for in-context recall, models are tasked with autoregressively predicting the underlined values in this example.

In the baseline setting of this task, we use a vocabulary of 16 tokens, which are equally divided into keys and values, 12,800 training sequences with a length of 128 tokens, and a share of 20% noise tokens in the input from a separate noise vocabulary of size 16.

#### E.1.4 Selective Copying

The selective copying task comprises sequences of randomly sampled tokens, with randomly inserted "blank" and "insert" tokens:

input example: a c [b] t [b] [i] [i] [b] [i] | a c [b] t [b] a c  
[b] t

In this example, tokens are drawn from the vocabulary  $\{a, c, t\}$ , while [b] and [i] indicate the blank and insert token. Given this example, the task of the model is to copy all non-special tokens to the positions of the insert tokens, in the order they were presented in the sequence. The purpose of the randomly inserted blank tokens is to force models to learn to selectively memorize or ignore information from the input.

In the baseline setting of this task, models are tasked with copying 16 randomly drawn tokens from a vocabulary of 16 tokens, and are provided with 12,800 training sequences with a length of 256 tokens.

### E.1.5 Compression

The compression task consists of random token sequences, each ending with a dedicated "compression token":

input example: a e c b h g i [c] | [c] + [pos<sub>0</sub>] -> a

In this example, tokens are randomly drawn from the vocabulary  $\{a, b, c, e, g, h, i\}$ , while [c] indicates the compression token. Given this input, models are tasked with compressing all relevant sequence information into the compression token [c], such that a subsequent two-layer MLP can fully recover each token of the input sequence, given the model's output for the compression token. To indicate the position  $i$  that is to be recovered from the input, we add a non-learnable sin-cos position embedding (indicated by [pos <sub>$i$</sub> ]) to the model's output for the compression token before feeding it to the MLP decoder.

In the baseline setting of this task, we use a vocabulary of 16 tokens and 12,800 training sequences with a length of 32 tokens.

### E.1.6 Memorization

The memorization task uses a fixed key-value dictionary, representing the facts to be learned:

key-value dictionary example:  $\{a:b, c:d, e:f\}$

Input sequences comprise key-value pairs that are randomly sampled from this dictionary. Importantly, all values are masked out from the input sequences with a dedicated "insert token":

input example: a [i] c [i] e [i] a [i] | a b c d e f a b

In this example, the values that are to be inserted at the positions of the insert tokens are: 'b', 'd', 'f', and 'b'. Models are then tasked with correctly inserting the masked-out values at the positions of the insert tokens. As the values are never part of the input sequences, models need to learn the mapping from keys to values over the course of their training.

In the baseline setting of this task, we use a vocabulary of 256 tokens, equally divided into keys and values, and 256 training sequences with a length of 32 tokens (such that each fact is on average presented 32 times in the training data).

## F Ablations with H-Net tiny

We conducted an ablation study using an “H-Net Tiny” configuration trained on a subset of Wikipedia. This setup allows us to compare H-Net directly against a standard Transformer baseline under controlled, resource-constrained conditions. By training both models on raw byte-level data, we effectively isolate the impact of H-Net’s learned chunking mechanism versus the fixed processing of a standard Transformer.

### F.1 H-Net Tiny Construction

The H-Net Tiny model is constructed as a scaled-down version of the full H-Net architecture, designed to be trainable on consumer-grade hardware while retaining the core hierarchical characteristics.

- **Architecture:** It features a single-stage hierarchy with an encoder, a dynamic chunking layer, a main network, and a decoder.
- **Hyperparameters:**
  - Model Dimension ( $d_{model}$ ): 256
  - Heads ( $n_{heads}$ ): 4
  - Feed-forward Dimension ( $d_{ff}$ ): 1024
  - Sequence Length: 512 bytes
  - Layers: 2 Encoder, 4 Main, 2 Decoder
- **Dynamic Chunking:** The model employs a dynamic chunking module with a target compression ratio of  $N = 6$ , effectively learning to group bytes into larger semantic units.

This construction serves as an accurate modeling of the original H-Net [10] in a miniature setting, allowing us to observe the emergence of chunking behaviors without the computational cost of full-scale training.

### F.2 Wikipedia Training as Ablation

Training on a subset of Wikipedia serves as an ablation study for the in-context learning capabilities. Unlike synthetic tasks where structure is rigid, natural language requires the model to infer flexible boundaries (words, subwords, phrases). By comparing H-Net to a byte-level Transformer on this data:

1. We test if **dynamic chunking** provides a better inductive bias for character-level language modeling than standard attention over all characters.
2. We assess if the hierarchical structure helps in recalling sequences (In-Context Recall) even with limited training data.

### F.3 Results Comparison

We trained both H-Net Tiny and a baseline Transformer (with identical dimensions but without chunking layers) on the same Wikipedia subset. We then evaluated them on a recall task where the models had to complete a text sequence given a prefix.

Table 3: Comparison of H-Net Tiny vs. Transformer Baseline on Wikipedia Recall Task

| Metric                     | H-Net Tiny | Transformer Baseline |
|----------------------------|------------|----------------------|
| Average Accuracy           | 15.80%     | 17.30%               |
| Average Jaccard Similarity | 0.3979     | 0.1480               |

The results (Table 3) reveal an interesting trade-off. The **Transformer baseline** achieved a slightly higher raw byte-level accuracy (17.30% vs. 15.80%). However, **H-Net Tiny** demonstrated a significantly higher **Jaccard Similarity** (0.3979 vs. 0.1480).

This disparity suggests that while the Transformer may be better at guessing specific common characters or collapsed patterns (resulting in sparse hits), H-Net captures the *distribution* and *content* of the target sequence much better. The high Jaccard score indicates that H-Net retrieves the correct set of characters (the "bag-of-bytes" content) even if the exact sequential ordering is imperfect due to the limited training regime. This aligns with the hypothesis that H-Net’s dynamic chunking allows it to model larger semantic units, preserving content information more effectively than a flat byte-level Transformer, which struggles to maintain coherence over long raw-byte sequences.

For the H-Net model trained on a subset of Wikipedia data, we also record the **Ratio Loss**. To prevent trivial compression behaviors, H-Net employs a *ratio loss*, inspired by load balancing in Mixture-of-Experts models. The loss encourages the model to retain a target fraction of vectors, avoiding both excessive retention and over-compression:

$$L_{\text{ratio}} = \frac{N}{N-1} \left( (N-1)FG + (1-F)(1-G) \right),$$

where  $F$  is the fraction of vectors actually selected and  $G$  is the average predicted boundary probability;  $N$  controls the target compression ratio. While  $F$  is non-differentiable,  $G$  allows gradient-based optimization, guiding the model toward  $F \approx G \approx 1/N$ . The final training objective combines the autoregressive prediction loss  $L_{\text{AR}}$  with the ratio loss:

$$L = L_{\text{AR}} + \alpha \sum_{s=0}^{S-1} L_{\text{ratio}}^s,$$

where  $\alpha = 0.03$  balances accuracy and compression. This enables content-adaptive compression, letting H-Net retain vectors based on semantic importance rather than fixed rules.

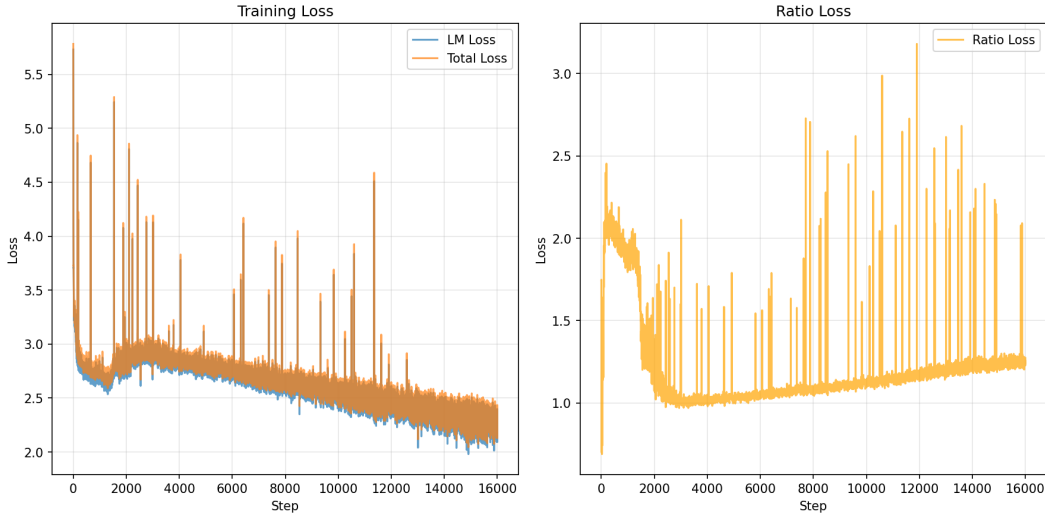


Figure 16: Training Loss for a 1-stage H-Net on a subset of Wikipedia, using the public H-Net demo configuration.

#### F.4 Qualitative Analysis: Wikipedia Subset

The following are the results of training H-Nets on a subset of Wikipedia. The model was trained on only 500 articles and 20 epochs, which limited its overall performance. Table 4 provides specific examples of model behavior.

We observe that while the H-Net output is syntactically degraded due to the limited training steps (e.g., generating repetitive patterns like "of the country of the contett"), the high Jaccard similarity indicates that the model successfully retrieves the correct *bag of characters* from the context. This suggests that the dynamic chunking mechanism prioritizes semantic content retention over local syntactic coherence in low-resource regimes.

- The model achieves an average accuracy of 15.80%, indicating limited recall capability after training.
- Test Case 6 shows the best performance (36%) when predicting structured date/name patterns.
- Test Case 9 demonstrates that high Jaccard similarity (0.6061) does not guarantee high accuracy (6%), suggesting the model captures character distributions but struggles with exact sequence prediction.
- The model appears to generate repetitive patterns (e.g., “of the country of the contett”) in several cases, indicating potential overfitting to certain phrases.

Table 4: Sample test Cases from H-net trained on Wikipedia subset

| ID | Accuracy | Jaccard | Matches | Target (first 30 chars)              | Predicted (first 30 chars)         |
|----|----------|---------|---------|--------------------------------------|------------------------------------|
| 1  | 20.00%   | 0.2500  | 10/50   | Though now an abugida, the Ge'ez     | Tn er                              |
| 2  | 10.00%   | 0.4400  | 5/50    | th). The intention was to exploit    | red Sn Maree) ad the saree of      |
| 3  | 8.00%    | 0.4667  | 4/50    | exico) Christian feast day: Adjutor  | oniaan) Tare of Ahmed (Inning      |
| 4  | 10.00%   | 0.3333  | 5/50    | ggravated assault. Simple assault    | oeeee th of tr of the sneee        |
| 5  | 20.00%   | 0.2273  | 10/50   | e distributed in Europe between 1996 | e aneeeeddddddd                    |
| 6  | 36.00%   | 0.6176  | 18/50   | ucer (b. 1918) 2013 – C. Gordon      | ucer (b. 192) 2000 – Jaa Barre     |
| 7  | 18.00%   | 0.3214  | 9/50    | of Ask and Embla in Völuspá provides | of the country of the contett      |
| 8  | 16.00%   | 0.5500  | 8/50    | re is to be launched by the end of   | te in the contrrtt of the contettt |
| 9  | 6.00%    | 0.6061  | 3/50    | anist and conductor 1945 – Rachel    | yy compaoser and politician 1950   |
| 10 | 14.00%   | 0.1667  | 7/50    | nm, suggestive of 6p to 7s transit   | m lis                              |